

实验：知识图谱可视化

【环境准备】：

Node 环境准备：

在 Windows 系统上，运行 nvm-setup.exe 安装 nvm，用于管理 Node 版本。

安装完成后，在终端输入如下指令，出现版本号则安装成功

```
nvm -v
```

输入 nvm list available 查看可用的 Node 版本，执行 nvm install [版本号] 即可安装对应版本的 node，这里安装 22.14.0 LTS 版本

```
nvm install 22.14.0
```

安装完成后，运行指令切换到该版本

```
nvm use 22.14.0
```

【任务 1】使用 ECharts 对知识图谱进行可视化

【ECharts 简介】

ECharts = Enterprise Charts，是由百度开源的一款纯 JavaScript 图表库，基于 Canvas 渲染，既能快速绘制常见统计图，也能胜任数十万数据点的复杂可视化。

ECharts 以**配置式**范式降低了可视化门槛，同时保持高性能、强扩展能力。
掌握套路：

1. 确定维度 → 2. 准备数据 → 3. 编写 option → 4. 交互&主题 → 5. 性能调优。

配合现代框架、TypeScript 与 Lottie 等动画资源，ECharts 可从 BI 仪表盘走向 WebGL 大屏乃至移动端小程序。

1. 核心优势

优势	说明
渲染性能	默认 Canvas，也支持 SVG 渲染（ <code>renderer: 'svg'</code> ），百万数据秒级渲染。
丰富的图表类型	30+ 种类：柱状、折线、饼图，到地理、关系、桑基、3D...
响应式	自动监听窗口缩放，也可手动调用 <code>chart.resize()</code> 。
数据驱动	所有可视元素皆由配置项驱动，易读易写。
生态完善	VS Code 插件、主题商店、社区扩展、TypeScript 声明文件。

2. 安装与引入

2.1 CDN 引入（最快体验）

```
html

<script src="https://cdn.jsdelivr.net/npm/echarts@5"></script>
<div id="chart" style="width:600px;height:400px;"></div>
<script>
  const chart = echarts.init(document.getElementById('chart'));
  chart.setOption({
    xAxis: { type: 'category', data: ['Mon', 'Tue', 'Wed', 'Thu', 'Fri'] },
    yAxis: { type: 'value' },
    series: [{ type: 'bar', data: [23, 45, 56, 12, 44] }]
  });
</script>
```

2.2 NPM 安装（推荐项目化）

bash

复制

```
npm i echarts --save
```

js

复制

```
import * as echarts from 'echarts';
```

3. 配置项总览

ECharts 配置对象简称 **option**，核心分区：

1. 全局：backgroundColor, title, legend, tooltip, grid...
2. 维度：xAxis, yAxis, angleAxis, radiusAxis, geo...
3. 数据：series 数组（每个成员就是一个图层）。
4. 动画与交互：animation, toolbox, dataZoom, brush, graphic...

js

复制

```
const option = {  
  title: { text: '销售趋势' },  
  tooltip: { trigger: 'axis' },  
  legend: { data:['2023'] },  
  xAxis: { type:'category', data:['1月','2月','3月','4月'] },  
  yAxis: { type:'value' },  
  series:[{  
    name:'2023',  
    type:'line',  
    data:[120, 200, 150, 80],  
    smooth:true  
  }]  
};
```

4. 常用图表速查

图表	type	关键配置
折线图	'line'	smooth, areaStyle 实现面积图
柱状图	'bar'	stack, barWidth
饼图	'pie'	radius, roseType
散点图	'scatter'	symbolSize
仪表盘	'gauge'	min, max, pointer
地图	'map'	需注册 GeoJSON: echarts.registerMap()

5. 与框架集成

Vue3（组合式）：

```
vue
复制

<script setup>
import * as echarts from 'echarts';
import { onMounted, ref, onBeforeUnmount } from 'vue';
const dom = ref(null);
let chart;
onMounted(()=>{ chart = echarts.init(dom.value); chart.setOption(option); });
onBeforeUnmount(()=> chart.dispose());
</script>

<template><div ref="dom" style="height:400px"/></template>
```

6. 资源

官网文档：<https://echarts.apache.org>

【安装 node 依赖包】

打开 cmd 命令行窗口，进入 project1-echarts

执行命令：

```
npm install
```

```
D:\Temp\kg_vis\project1-echarts>npm install

up to date, audited 51 packages in 2s

6 packages are looking for funding
  run `npm fund` for details

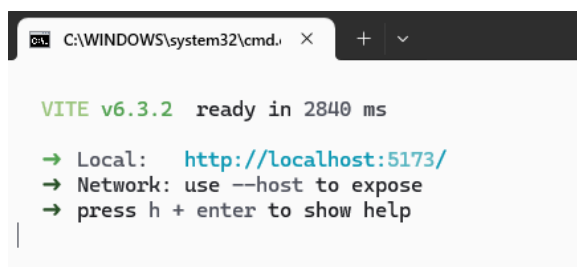
found 0 vulnerabilities
```

【运行项目】

打开 cmd 命令行窗口，进入 project1-echarts

执行命令：

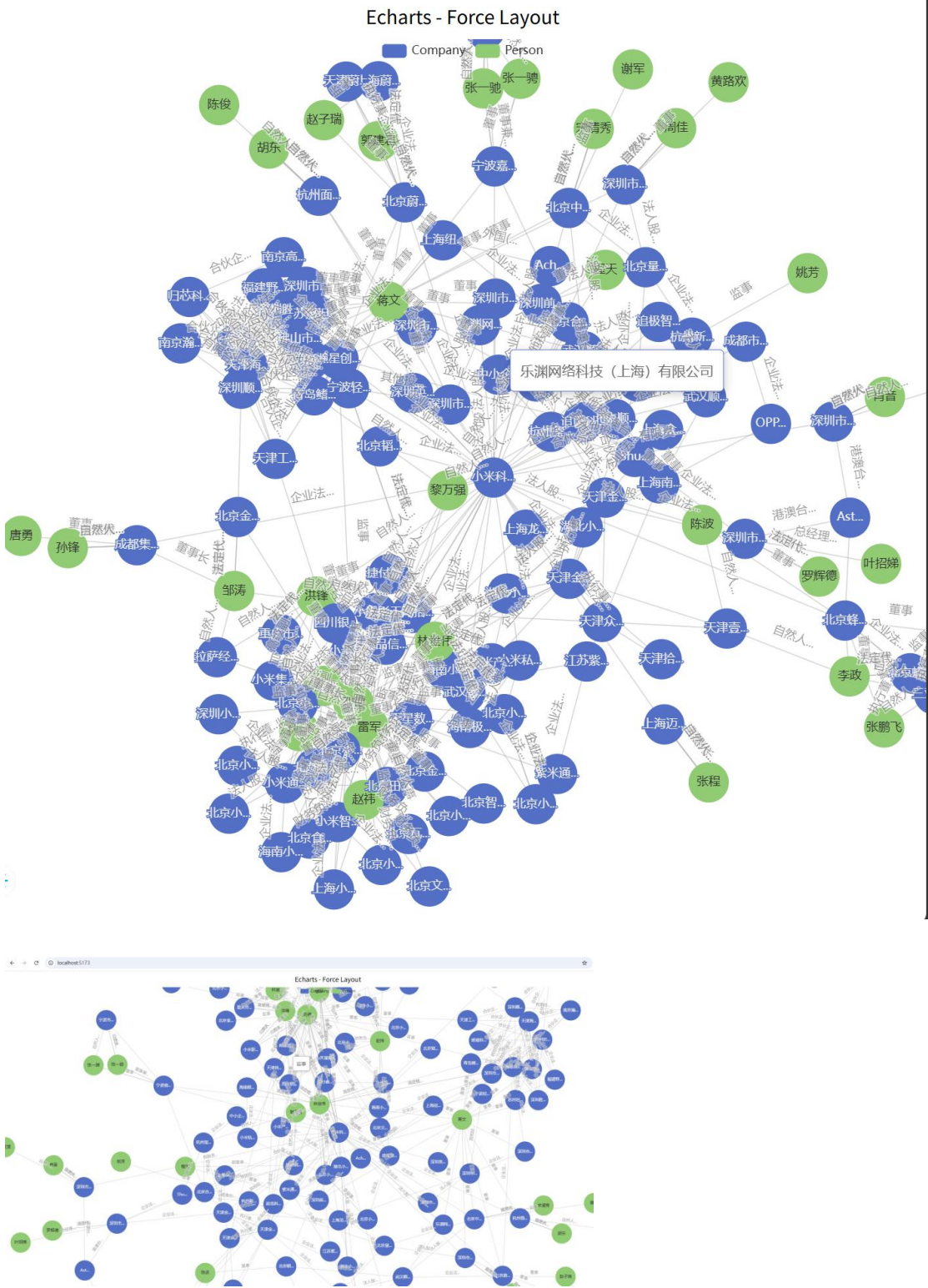
```
npm run dev
```



在浏览器中，访问 <http://localhost:5173/> 运行项目

【粘贴截图】：请粘贴你运行上述项目的浏览器截图。

例如,



【任务 2】使用 AntV G6 对知识图谱进行可视化

【AntV G6 简介】

G6 = **Graph Visualization Grammar**，阿里巴巴 AntV 团队开源的关系/网络图渲染与分析库。定位“数据驱动的图可视化引擎”，主打灵活的 **渲染层（Canvas / WebGL / SVG）+ 语法层（Graph、Node、Edge、Combo 等）** 组合，适用于社交网络、流程拓扑、知识图谱、金融风控等场景。

1. 核心特性

特性	说明
多渲染后端	默认 Canvas，内置 WebGL 插件，支持 10 万点极速渲染。
语法分层	数据层→元素层→可视层，抽象“图语法”便于高度定制。
布局丰富	30+ 布局：力导向、层次、环形、网格、combo-force、DAG 等。
交互体系	内置缩放、拖拽、框选、节点鱼眼、边平滑等交互，支持自定义行为。
模块化 & TS	ES module + TypeScript 声明，友好 tree-shaking。
图分析	Pathfinding、圈检测、PageRank、Centrality 等基础算法。

2. 快速上手

最小示例

```
html

<script type="module">
  import G6 from 'https://cdn.jsdelivr.net/npm/@antv/g6@5+esm';

  const data = {
    nodes:[{id:'Alice'},{id:'Bob'}],
    edges:[{source:'Alice',target:'Bob'}]
  };

  const graph = new G6.Graph({
    container: 'mount',
    width: 600,
    height: 400,
    layout: { type:'force' },
    defaultNode:{ size:40, style:{ fill:'#6ea6ff' }},
    defaultEdge:{ style:{ endArrow:true }}
  });

  graph.data(data);
  graph.render();
</script>
```

3. 数据与元素模型

```
Graph
├─ Node (节点)
├─ Edge (边, 支持多边/自环)
└─ Combo (组合, 可折叠子图)
```

数据格式:

```
{
  nodes: [{ id, label, ...attrs }],
  edges: [{ id, source, target, ...attrs }],
  combos: [{ id, parentId, ...attrs }]
}
```

4. 布局体系

- 力导向 (force/gForce WebGL 加速)
- 层次 (dagre、indented) 适合流程图
- 圆 / 椭圆 (circular)
- 网格 (grid)
- 复合力导向 (comboForce) 支持 combo
- 连续布局动画: layout.updateCfg({ center: [...] })

可在 run 时动态切换:

```
graph.updateLayout({ type:'dagre', rankdir:'LR' });
```

5. 渲染与样式

5.1 Shape 绘制

G6 内置 shape: circle, rect, diamond, ellipse, polyline 等。

```
G6.registerNode('card-node', {
  draw(cfg, group) {
    const keyShape = group.addShape('rect', {
      attrs:{ width:180, height:60, fill:'#fff', stroke:'#5B8FF9', radius:4 }
    });
    group.addShape('text',{
      attrs:{ x:10, y:20, text: cfg.label, fill:'#000', fontSize:14 }
    });
    return keyShape; // 返回关键形, 用于拾取
  }
}, 'single-node');
```


5.2 主题

```
graph.setTheme({
  node:{ style:{ fill:'#F6F8FA', stroke:'#D9D9D9' }},
  edge:{ style:{ stroke:'#A3B1BF' }}
});
```

6. 交互机制

- 内置 行为 (Behavior) : drag-node, zoom-canvas, drag-canvas, activate-relations, brush-select ...
- 事件冒泡: node:click, edge:mouseenter, canvas:contextmenu
- 自定义行为:

```
G6.registerBehavior('collapse-expand', {
  getEvents() { return { 'node:dblclick':'onDbl' }; },
  onDbl(e) {
    const item = e.item;
    graph.collapseExpandCombo(item); // 折叠/展开
  }
});
graph.addBehaviors(['collapse-expand'], 'default');
```

- 官网 & 文档: <https://g6.antv.antgroup.com>
- 示例库: <https://g6.antv.antgroup.com/examples>

【安装 node 依赖包】

打开 cmd 命令行窗口, 进入 project2-g6

执行命令:

```
npm install
```

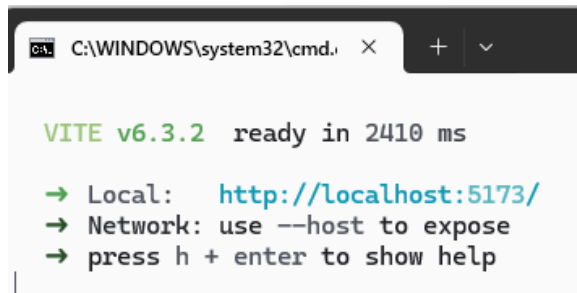
```
D:\Temp\kg_vis\project2-g6>npm install
up to date, audited 212 packages in 11s
16 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
```

【运行项目】

打开 cmd 命令行窗口，进入 project2-g6

执行命令：

```
npm run dev
```

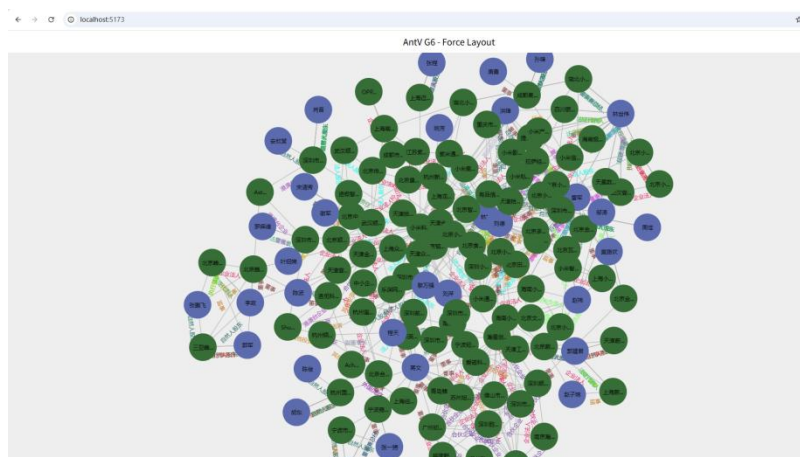
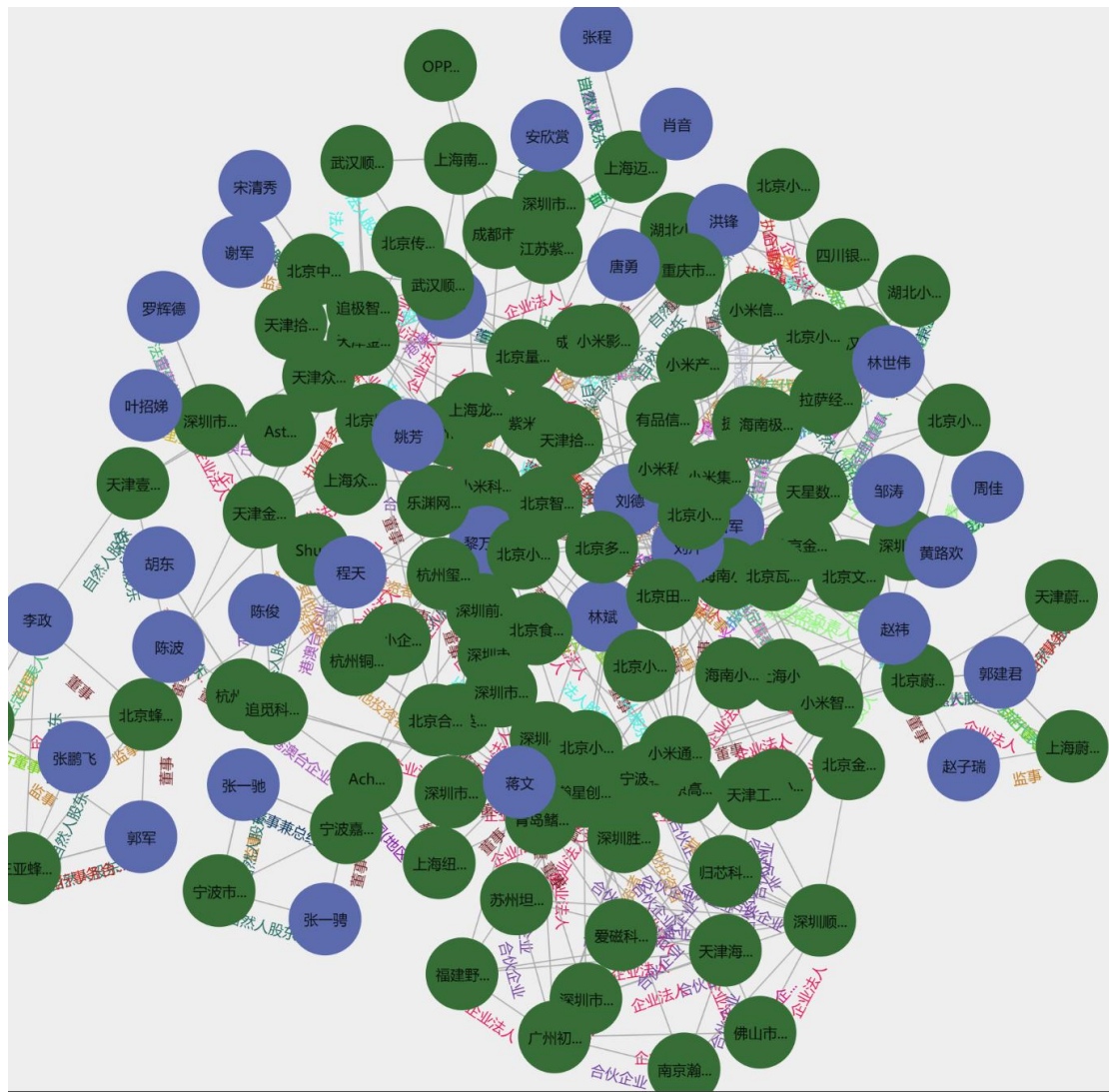


```
C:\WINDOWS\system32\cmd.exe X + v  
  
VITE v6.3.2 ready in 2410 ms  
→ Local:   http://localhost:5173/  
→ Network: use --host to expose  
→ press h + enter to show help
```

在浏览器中，访问 <http://localhost:5173/> 运行项目

【粘贴截图】：请粘贴你运行上述项目的浏览器截图。

例如,



【任务 3】使用 D3.js 对知识图谱进行可视化

【D3.js 简介】

D3 = Data-Driven Documents。由 Mike Bostock 2011 年开源的 **原生 JavaScript 可视化引擎**，通过“把数据映射到 DOM / SVG / Canvas / WebGL 属性”实现任意图形，是现代可视化框架（ECharts、G2、Vega、Observable Plot…）的底层灵感来源。

1. 核心理念

关键词	释义
Selection	用 CSS 选择器抽取 DOM 节点，原生 API 友好。
Data Binding	<code>selection.data()</code> 将数组映射到元素，数据驱动视图。
Enter-Update-Exit	针对新增、更新、删除三类数据生成对应节点，动画无痛。
过渡 (Transition)	对任何可动画属性插值，保真到帧。
组件化	50+ 独立模块 (scale、shape、geo、force...)，可按需引入。
功能即库	只提供底层 <i>lambda</i> ，不帮你画“折线”或“饼图”，完全自由。

2. 快速上手

“Hello D3” —— 10 行折线图

html

复制

```
<svg id="chart" width="500" height="300"></svg>
<script src="https://cdn.jsdelivr.net/npm/d3@7"></script>
<script>
  const data = [23, 65, 18, 90, 46];
  const svg = d3.select('#chart');
  const x = d3.scaleLinear().domain([0, data.length - 1]).range([40, 460]);
  const y = d3.scaleLinear().domain([0, d3.max(data)]).range([260, 20]);

  const line = d3.line((d, i) => x(i), d => y(d)).curve(d3.curveCatmullRom);

  svg.append('path')
    .datum(data) // 绑定整组数据
    .attr('fill', 'none')
    .attr('stroke', '#5B8FF9')
    .attr('stroke-width', 2)
    .attr('d', line); // 由数据 → “d” 路径字符串

  svg.append('g').attr('transform', 'translate(0,260)').call(d3.axisBottom(x));
  svg.append('g').attr('transform', 'translate(40,0)').call(d3.axisLeft(y));
</script>
```

3. 重要模块概览

模块	常用方法	典型用途
d3-selection	<code>select</code> , <code>selectAll</code> , <code>append</code> , <code>attr</code> , <code>style</code>	操作 DOM
d3-scale	<code>scaleLinear</code> , <code>scaleBand</code> , <code>scaleOrdinal</code>	映射尺寸、颜色
d3-axis	<code>axisBottom</code> , <code>axisLeft</code>	生成坐标轴
d3-shape	<code>line</code> , <code>area</code> , <code>arc</code> , <code>pie</code>	构造路径数据
d3-transition	<code>transition</code> , <code>duration</code> , <code>ease</code>	插值动画
d3-array	<code>extent</code> , <code>sum</code> , <code>rollup</code> , <code>histogram</code>	数据统计
d3-force	<code>forceSimulation</code> , <code>forceLink</code> , <code>forceManyBody</code>	力导向网络图
d3-geo	<code>geoPath</code> , <code>geoMercator</code> , <code>geoNaturalEarth1</code>	地图投影
d3-zoom	<code>zoom</code> , <code>zoomTransform</code>	平移缩放交互
d3-fetch	<code>csv</code> , <code>json</code> , <code>tsv</code>	异步加载数据

4. 布局与算法

- d3-force: 力导向关系图
- d3-hierarchy: treemap, pack, partition, cluster 处理树/层级
- d3-chord / d3-ribbon: 弦图
- d3-voronoi / d3-delaunay: 空间剖分
- d3-sankey: 桑基流（官方外部插件）

官方 API: <https://github.com/d3/d3/blob/main/API.md>

【安装 nginx】

下载 nginx

<https://nginx.org/en/download.html>

nginx: download

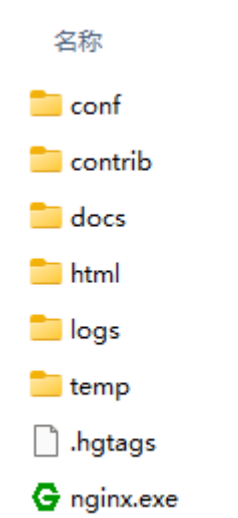
Mainline version

[CHANGES](#) [nginx-1.27.5](#) [pgp](#) [nginx/Windows-1.27.5](#) [pgp](#)

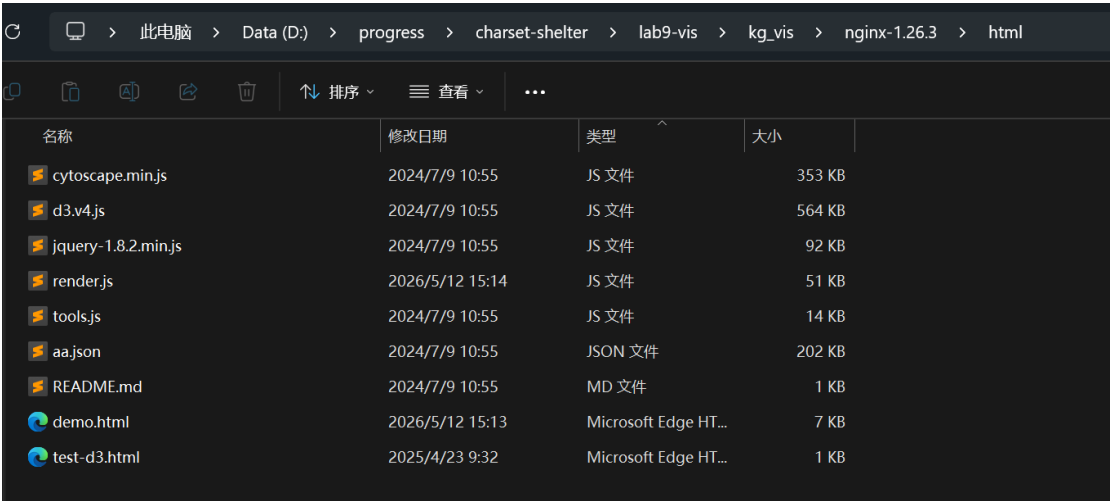
Stable version

[CHANGES-1.26](#) [nginx-1.26.3](#) [pgp](#) [nginx/Windows-1.26.3](#) [pgp](#)

解压 nginx 压缩包



将 project3-d3 目录中的内容（不包含 project-d3 本身）复制到 nginx 的 html 目录下，并修改 conf/nginx.conf 文件。



```
server {
    listen      80;
    server_name localhost;

    #charset koi8-r;

    #access_log logs/host.access.log main;

    location / {
        root    html;
        index   demo.html;
    }
}
```


以管理员身份打开命令提示符 cmd，转到 nginx 目录下。

执行下面命令，启动 nginx：

```
start nginx.exe
```

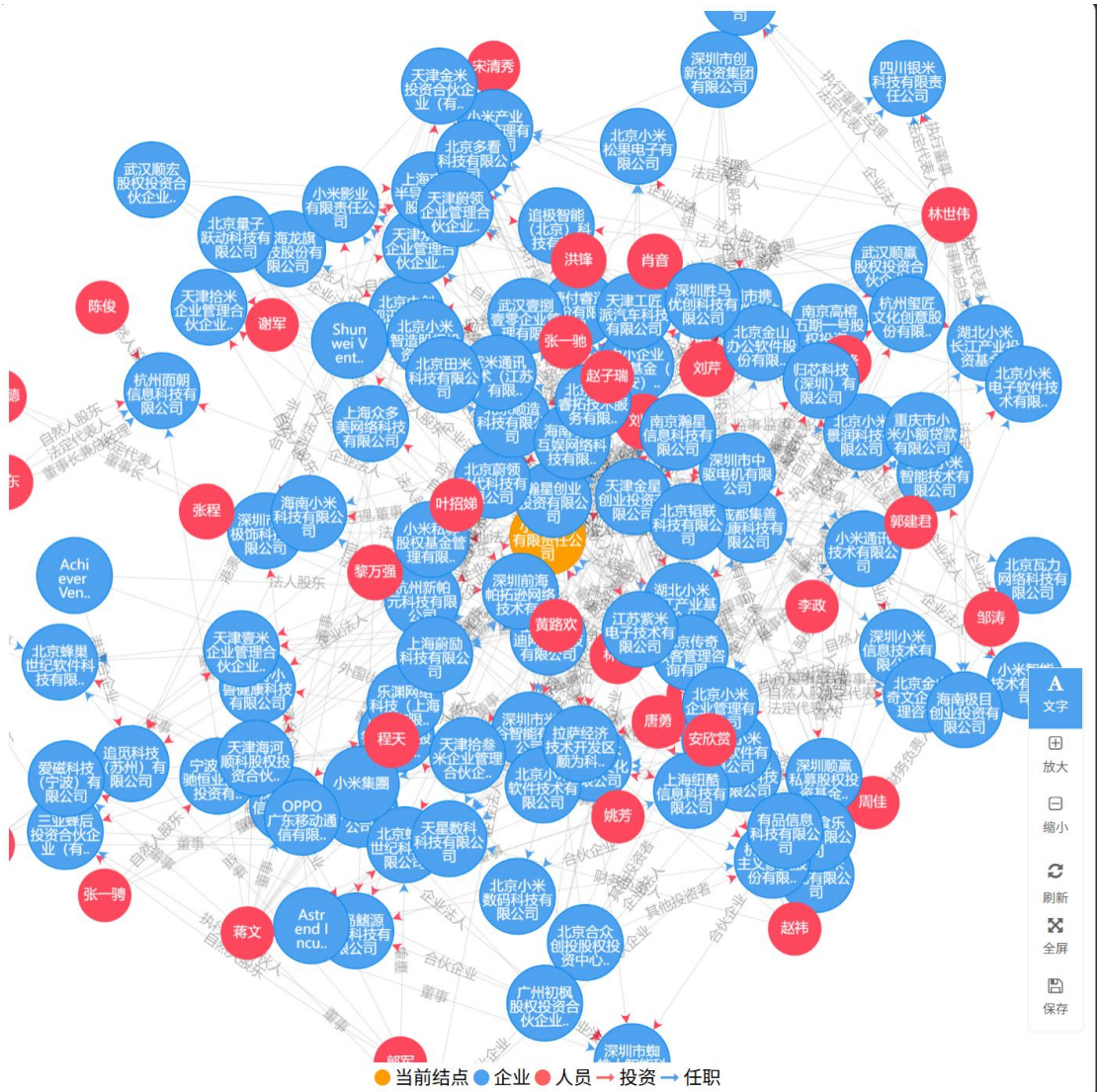
如要停止 nginx，使用以下命令：

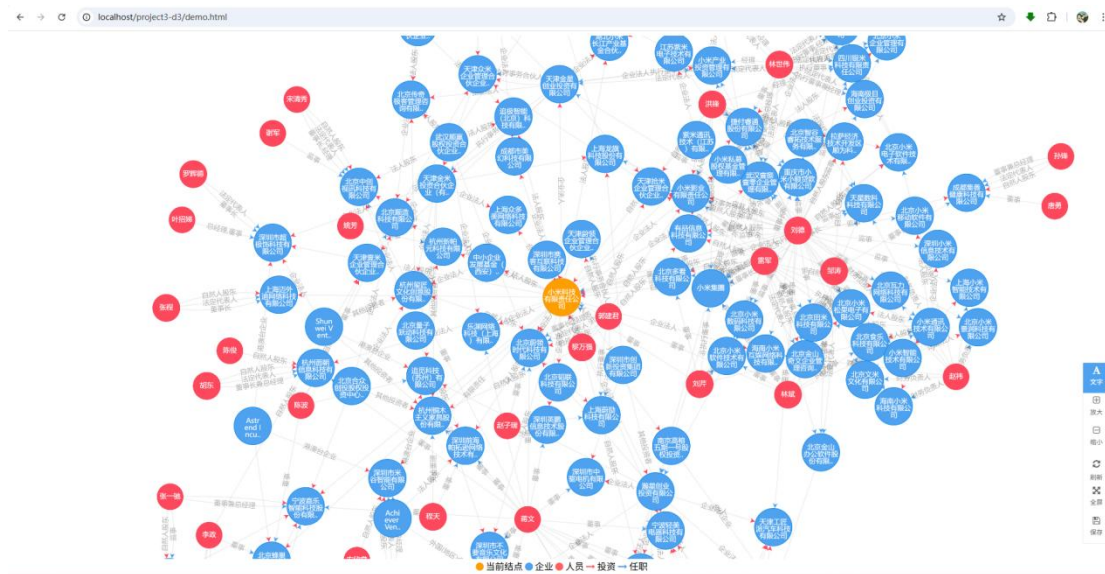
```
nginx.exe -s stop
```

访问 <http://localhost/demo.html> 即可看到知识图谱

【粘贴截图】：请粘贴你运行上述项目的浏览器截图。

例如，



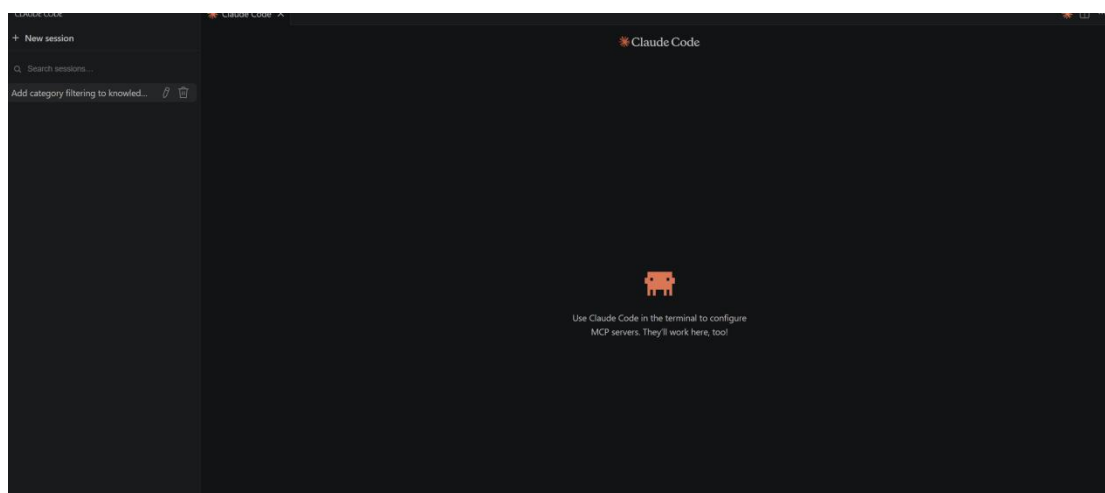


【任务 4】利用 vibe coding 实现功能添加

近年来，随着 LLM 的发展，AI 辅助编程成为了开发中的一项重要方法，各大服务提供商也提供了相应的工具与模型，用于 AI 辅助编程。其中，vibe coding 作为一种新型的辅助编程方法，其可以使开发者仅关注实现效果而非具体实现，通过自然语言的方式以近乎零代码或者少量代码编写的消耗，完成开发任务。vibe coding 全称“氛围编程”，意为开发者可以投入到“开发氛围”而非开发实现。在本任务中，我们将基于 vibe coding 的方式，为以上三个任务添加 3 种不同的功能。

在本指导书中，我们将以 claude code 作为 agent 服务提供商为例，演示如何使用 vibe coding。当然，也可以按照需要选择其他类型的 agent 服务商，如 codex、augment code 和 LLM 厂商提供的 coding 平台等。首先，安装 claude code，并在 vscode 中配置好 claude code 插件，配置好 base_url 和 API key。

如果无法使用 claude code、codex 等工具，这里推荐一个不需要科学上网且免费的 vibe coding 工具：TRAE，vscode 安装插件 TRAE AI: Coding Assistant 即可使用。



【任务 4-1】为任务 1 添加“分类展示功能”

在任务中，我们将完成如下功能的添加：在图谱系统中添加一个菜单，用户可以选择任意一种实体类型，系统将会展示该类型下的所有实体。若该类别实体之间存在边相连，则也需要显示对应的边。

在插件中点击 new session，创建新的上下文，并输入新功能的描述和实现需求。

```
Modify the project in the project1-echarts folder, to add a new feature:  
User can select one of the category of the knowledge graph, and the system only shows the nodes of the selected category. When these nodes are connected by the edges, show the edges as well.
```

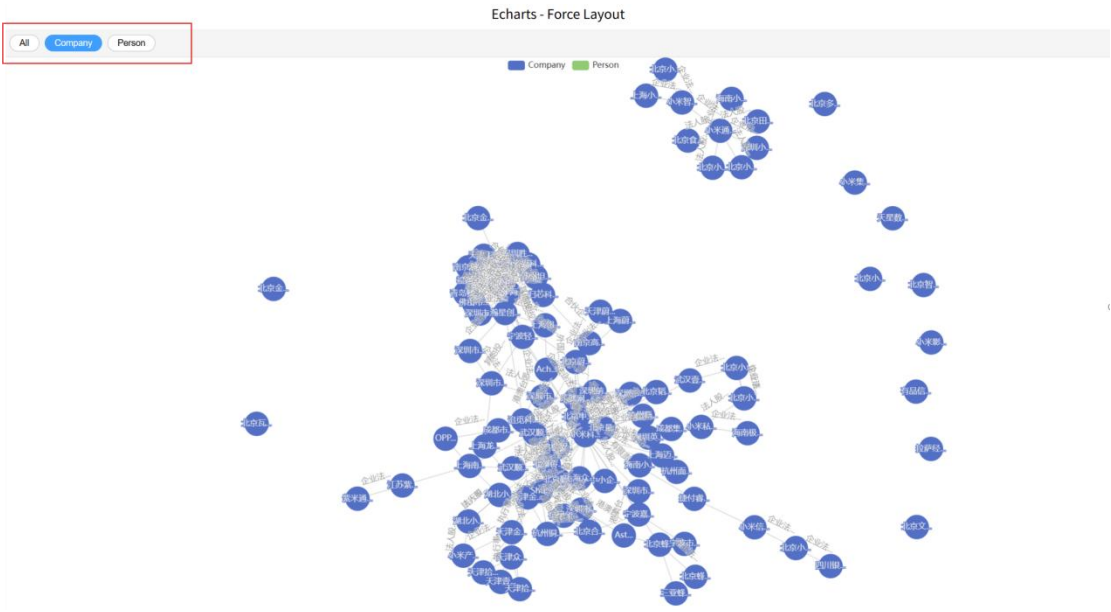
- Thought for 31s >
- Let me explore the project structure first.
- Glob** pattern: "**/*"
Found 101 files
- Read** d:\progress\charset-shelter\lab9-vis\kg_vis\project1-echarts\src\App.vue
- Read** d:\progress\charset-shelter\lab9-vis\kg_vis\project1-echarts\src\components\Graph.vue
- Read** d:\progress\charset-shelter\lab9-vis\kg_vis\project1-echarts\src\data\realData.json
- Read** d:\progress\charset-shelter\lab9-vis\kg_vis\project1-echarts\src\utils\color.js
- Thought for 35s >
- Now let me check the base styles and HTML structure.
- Read** d:\progress\charset-shelter\lab9-vis\kg_vis\project1-echarts\src\style.css
- Read** d:\progress\charset-shelter\lab9-vis\kg_vis\project1-echarts\index.html
- Thought for 8s >
- I have a clear picture of the codebase. Now I'll update `Graph.vue` to add category filtering with buttons, filtered nodes, and filtered edges.
- Write** d:\progress\charset-shelter\lab9-vis\kg_vis\project1-echarts\src\components\Graph.vue
162 lines

```
<template>  
  <div class="graphWrapper">  
    <div class="categoryFilter">
```

选择“允许 agent 修改文件”，等待其修改完成。

```
const links = rawData.relationships.map((rel) => ({  
  label: {  
    formatter: (rel.properties.name.length > 3) ? rel.properties.name.slice(0, 3) : rel.properties.name,  
  },  
  lineStyle: {  
    color: '#aaaa',  
  },  
}))  
  
// 提取唯一的分类信息  
const categories = [...new Set(nodes.map((n) => n.category))].map((name) => {  
  name,  
}))  
  
const initChart = () => {  
  if (!echartRef.value) return  
  if (chartInstance) chartInstance.dispose()  
  
  chartInstance = echarts.init(echartRef.value)  
  
  const option = {  
  
    tooltip: {  
      formatter: (params) => {  
        return params.data.name  
      }  
    }  
  }  
  
  const allLinks = rawData.relationships.map((rel) => ({  
    label: {  
      formatter: (rel.properties.name.length > 3) ? rel.properties.name.slice(0, 3) : rel.properties.name,  
    },  
    lineStyle: {  
      color: '#aaaa',  
    },  
  }))  
  
  const categoryNames = [...new Set(allNodes.map((n) => n.category))]  
  const categories = categoryNames.map((name) => ({ name })))  
  
  const getFilteredData = () => {  
    if (selectedCategory.value === 'All') {  
      return { nodes: allNodes, links: allLinks }  
    }  
    const filteredNodes = allNodes.filter((n) => n.category === selectedCategory.value)  
    const nodeIds = new Set(filteredNodes.map((n) => n.id))  
    const filteredLinks = allLinks.filter((l) => nodeIds.has(l.source) && nodeIds.has(l.target))  
    return { nodes: filteredNodes, links: filteredLinks }  
  }  
  
  const updateChart = () => {  
    if (!chartInstance) return  
    const { nodes, links } = getFilteredData()  
  
    chartInstance.setOption({  
      tooltip: {  
        formatter: (params) => params.data.name  
      }  
    })  
  }  
}
```

与任务 1 一致，运行该系统，可以看到在上方出现了选择按钮。点击"Company"按钮，可以看到所有该类别的实体被筛选出来，并展示出了类别内部相连的边。当然，也可以进一步优化样式，使其效果更加美观。



【任务 4-2】为任务 2 添加“统计信息和 PageRank 算法”

与任务 4-2 类似，我们同样可以为任务 2 新增一个功能：在侧边栏展示图谱统计信息，并提供一个 PageRank 算法的运行按钮。将需求发送给 claude code，并等待其运行完成。

```
Very well, now switch to the project in the project2-g6 folder. Add a feature:
In the sidebar, user may view the general metric of the graph. And the user can run the Pagerank algo for the whole graph. The sidebar will not show when the mouse is not hovered.

• Thought for 134s >
• Let me read the project2-g6 files to understand the current structure.
• Read d:\progress\charset-shelter\lab9-vis\kg_vis\project2-g6\src\App.vue
• Read d:\progress\charset-shelter\lab9-vis\kg_vis\project2-g6\src\components\Graph.vue
• Read d:\progress\charset-shelter\lab9-vis\kg_vis\project2-g6\src\data\realData.json
• Read d:\progress\charset-shelter\lab9-vis\kg_vis\project2-g6\src\utils\color.js
• Thought for 124s >
• Let me check the full data file to understand the graph size and structure.
• Bash
  IN  node -e "
    const data = require('./project2-g6/src/data/realData.json');
    console.log('nodes:', data.nodes.length);

  OUT nodes: 138
    edges: 384
    categories: [ 'Company', 'Person' ]

• Thought for 84s >
• Read d:\progress\charset-shelter\lab9-vis\kg_vis\project2-g6\package.json
```

```

const n = processedNodes.length

const outDegree = {}
const inNeighbors = {}
processedNodes.forEach((node) => {
  outDegree[node.id] = 0
  inNeighbors[node.id] = []
})
processedEdges.forEach((edge) => {
  outDegree[edge.source] = (outDegree[edge.source] || 0) + 1
  if (inNeighbors[edge.target]) inNeighbors[edge.target].push(edge.source)
})

let ranks = {}
processedNodes.forEach((node) => { ranks[node.id] = 1 / n })

for (let i = 0; i < iterations; i++) {
  const next = {}
  processedNodes.forEach((node) => {
    let sum = 0
    inNeighbors[node.id].forEach((src) => {
      sum += ranks[src] / (outDegree[src] || 1)
    })
    next[node.id] = (1 - d) / n + d * sum
  })
  ranks = next
}

const scored = processedNodes
  .map((node) => ({ id: node.id, name: node.data.name, score: ranks[node.id] }))
  .sort((a, b) => b.score - a.score)

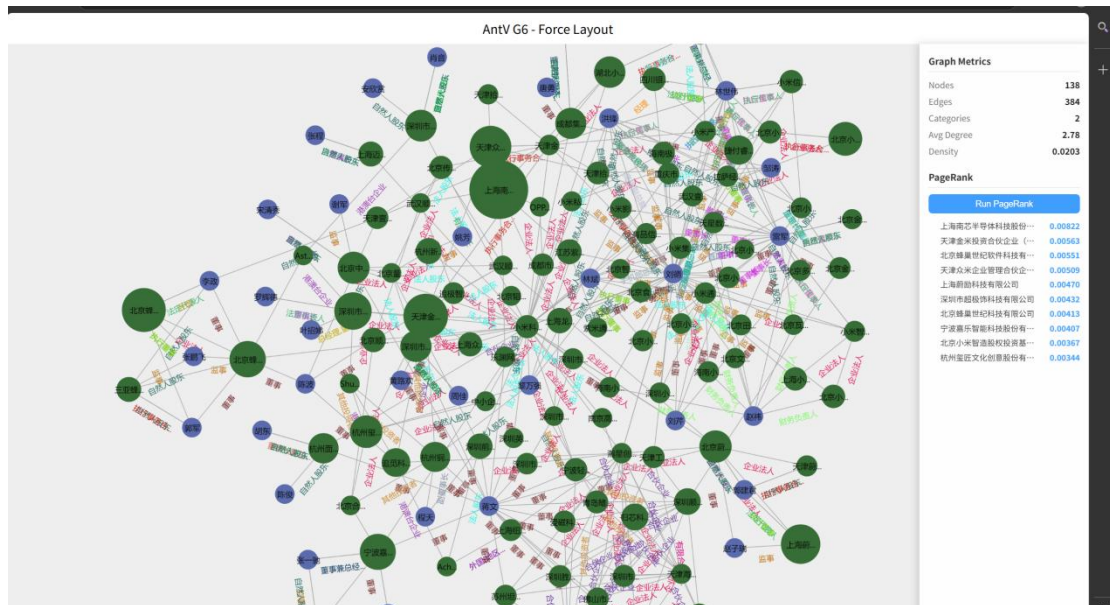
pagerankResults.value = scored.slice(0, 10)

const maxScore = scored[0].score
const minScore = scored[scored.length - 1].score
const range = maxScore - minScore || 1

const updates = processedNodes.map((node) => ({
  id: node.id,
  data: { pagerankNorm: (ranks[node.id] - minScore) / range },
}))
graphInstance.updateNodeData(updates)
graphInstance.draw()
pagerankRunning.value = false
}

```

运行该系统，可以看到右侧已经出现了相应的侧边栏。



【任务 4-3】为任务 3 添加节点度数统计和排序

与以上两个任务类似，我们为任务 3 添加新的功能：在侧边栏展示节点的度数统计。claude code 具有上下文记忆功能，如图所示，即使在中途任务中断，也可以指示模型继续完成现有任务，恢复工作。

```
Very well, now switch to the project3-d3 folder, and add a new feature: counting the degrees of the nodes and draw the bar graph. Besides, recognize the "key" node of the graph.

• Thinking >
• Let me read all the relevant files in project3-d3.
• Read d:\progress\charset-shelter\lab9-vis\kg_vis\project3-d3\demo.html, d:\progress\charset-shelter\lab9-vis\kg_vis\project3-d3\render.js
• Read d:\progress\charset-shelter\lab9-vis\kg_vis\project3-d3\tools.js, d:\progress\charset-shelter\lab9-vis\kg_vis\project3-d3\aa.json
• Thinking >
• Now I have a thorough understanding of the project. It's a vanilla HTML+JS project using Cytoscape.js for rendering and D3 v4 for force layout. Let me plan the implementation before writing code.
  Plan:
  1. demo.html — add analytics panel HTML (slides in from left on hover), CSS for the panel
  2. render.js — add computeDegrees, drawDegreeChart, drawKeyNodeList functions; call them after data loads; add .keynode Cytoscape style and apply it in cy.ready()
• Update Todos
  * Add analytics panel HTML + CSS to demo.html
  * Add degree computation and D3 bar chart functions to render.js
  * Wire up analytics call in getData and key node highlighting in cyready()
• No response requested.

Ok, continue this task.
```

```
// — analytics —

// Returns nodes sorted by total (undirected) degree, highest first
function computeDegrees(nodes, links) {
  var degMap = {};
  nodes.forEach(function(n) { degMap[n.nodeId] = 0; });
  links.forEach(function(l) {
    degMap[l.sourceNode.nodeId] = (degMap[l.sourceNode.nodeId] || 0) + 1;
    degMap[l.targetNode.nodeId] = (degMap[l.targetNode.nodeId] || 0) + 1;
  });
  return nodes.map(function(n) {
    return {
      nodeId: n.nodeId,
      name: n.data.obj.properties.name,
      type: n.data.obj.labels[0],
      degree: degMap[n.nodeId] || 0
    };
  }).sort(function(a, b) { return b.degree - a.degree; });
}

// Horizontal bar chart of top nodes by degree (D3 v4)
function drawDegreeChart(topNodes) {
  var el = document.getElementById('degreeChart');
  if (!el) return;
  el.innerHTML = '';

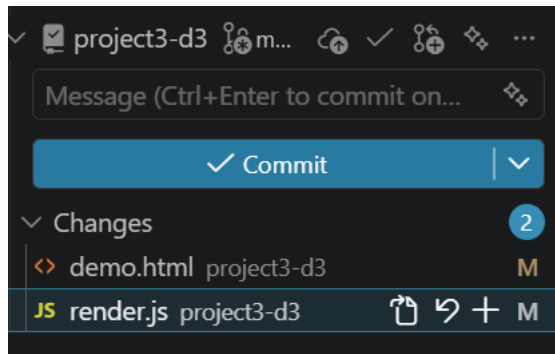
  var W = 226, mL = 72, mR = 32, mT = 8, mB = 20;
  var innerW = W - mL - mR;
  var barH = 20, gap = 5;
  var innerH = topNodes.length * (barH + gap);

  var svg = d3.select('#degreeChart').append('svg')
    .attr('width', W)
    .attr('height', innerH + mT + mB);

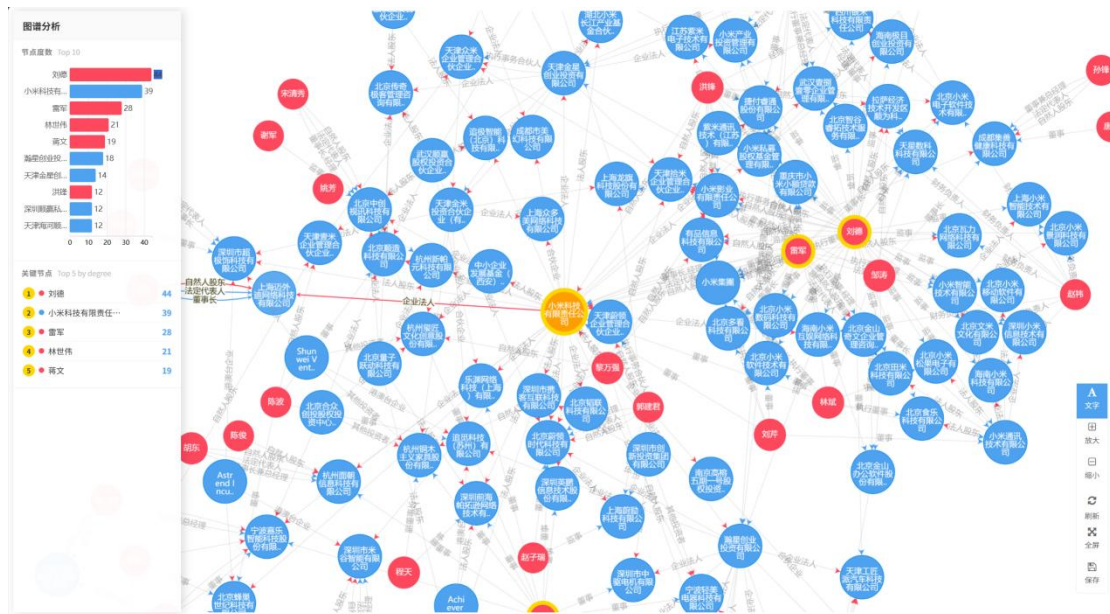
  var g = svg.append('g').attr('transform', 'translate(' + mL + ', ' + mT +

  var maxDeg = d3.max(topNodes, function(d) { return d.degree; }) || 1;
  var x = d3.scaleLinear().domain([0, maxDeg]).range([0, innerW]);

  // subtle vertical grid lines
  g.append('g')
    .attr('transform', 'translate(0, ' + innerH + ')')
    .call(d3.axisBottom(x).ticks(4).tickSize(-innerH).tickFormat(''))
    .select('.domain').remove();
}
```



运行系统，可以看到左侧的统计页面。



【粘贴截图】

自选 vide coding 平台，完成以上三个新功能的添加，要求截图 prompt 和运行过程，修改代码的 diff 记录，以及最终系统运行截图。

任务 4-1:



The screenshot shows a dark-themed interface for an AI agent. At the top left, there is a circular icon with a right-pointing arrow. Below it, the text 'Agent' is displayed next to a small icon of a screen with a grid. At the bottom left, there is a button labeled '思考过程' (Thinking Process) with a right-pointing arrow. The main content area is a light gray box containing a list of tasks in Chinese.

请为这个AntV G6知识图谱项目添加以下功能:

1. 在页面右侧添加一个固定宽度的侧边栏
2. 侧边栏顶部显示图谱统计信息: 总节点数、总边数、节点类型数量、平均度数
3. 添加一个"运行PageRank算法"按钮
4. 点击按钮后, 计算所有节点的PageRank值, 并将节点大小映射为PageRank值 (值越大节点越大)
5. 在侧边栏中显示PageRank值排名前10的节点列表
6. 保持原有的所有交互功能不变


```

<script setup>
import { ref, computed, provide } from 'vue'
import Graph from './components/Graph.vue'
import rawData from './data/realData.json'

const graphRef = ref(null)

const stats = computed(() => {
  const totalNodes = rawData.nodes.length
  const totalEdges = rawData.relationships.length
  const nodeTypes = new Set(rawData.nodes.map(n => n.labels[0] || 'Unknown')).size

  const degreeMap = new Map()
  rawData.relationships.forEach(rel => {
    degreeMap.set(rel.startNode, (degreeMap.get(rel.startNode) || 0) + 1)
    degreeMap.set(rel.endNode, (degreeMap.get(rel.endNode) || 0) + 1)
  })
  const totalDegree = Array.from(degreeMap.values()).reduce((sum, d) => sum + d, 0)
  const avgDegree = totalNodes > 0 ? (totalDegree / totalNodes).toFixed(2) : 0

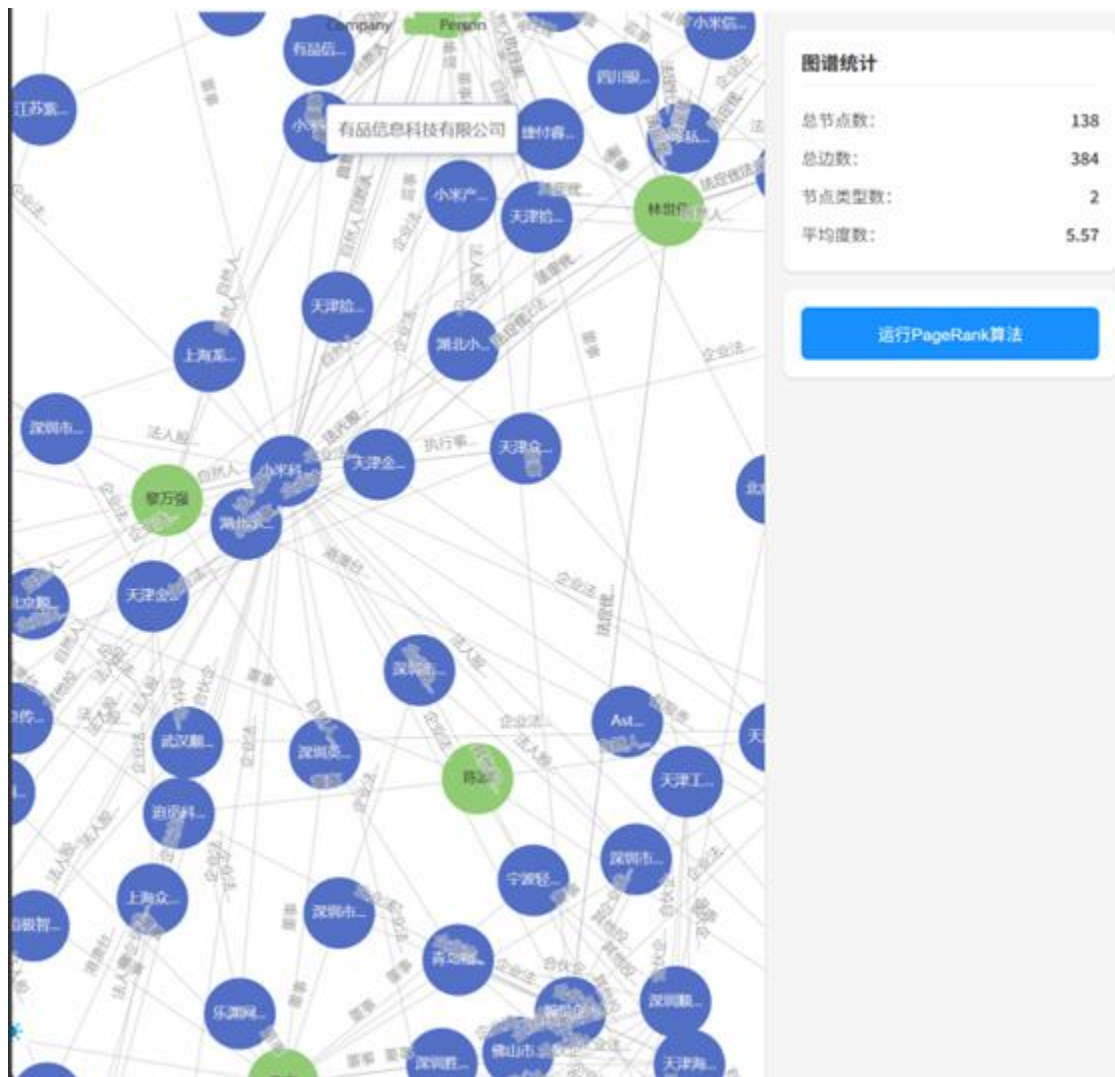
  return { totalNodes, totalEdges, nodeTypes, avgDegree }
})

const pageRankList = ref([])
const isPageRankRunning = ref(false)

const runPageRank = () => {
  if (graphRef.value) {
    isPageRankRunning.value = true
    const result = graphRef.value.runPageRank()
    pageRankList.value = result
    isPageRankRunning.value = false
  }
}

```

 是否要
(Offic



任务 4-2:

请为这个D3.js知识图谱项目添加节点度数统计功能:

1. 在页面左侧添加一个固定宽度的侧边栏
2. 侧边栏上半部分绘制节点度数分布直方图 (x轴为度数, y轴为对应节点数量)
3. 侧边栏下半部分按度数从高到低排序显示所有节点列表, 显示节点名称和度数
4. 点击列表中的节点时, 在图谱中高亮该节点及其相连的边
5. 保持原有的力导向布局和拖拽功能不变

```
import { ref, onMounted, defineExpose } from 'vue'
import data from '../data/realData.json'
import { stringToColor } from '../utils/color.js'

const graphData = JSON.parse(JSON.stringify(data))

graphData.nodes = graphData.nodes.map((node, index) => {
  return {
    id: node.id,
    data: {
      label: node.labels,
      ...node.properties,
    },
  },
})

graphData.relationships = graphData.relationships.map((relation, index) =>
  return {
    id: relation.id,
    source: relation.startNode,
    target: relation.endNode,
    data: { label: relation.properties.name, type: relation.type },
  }
})

const graph = ref(null)
const containerId = 'g6Container-' + Date.now()

onMounted(() => {
  graph.value = new Graph({
    autoFit: {
      type: 'view',
      options: {
        when: 'overflow',
      },
    },
  })
})
defineExpose({ graph })
```